

Advanced Algorithms and Data Structures

Marcello La Rocca

Foreword by Luis Serrano



MANNING

and in practice for $k \geq 7$ there is already no chance of having a dataset big enough to satisfy the inequality. For $n > 2^k$, however, this method still has an advantage over the brute-force search.

A final consideration: Pruning heavily depends on the “quality” of the nearest neighbor we have found so far. The shorter the distance to the current nearest neighbor, the more branches we can prune, and in turn the higher speed-up we can get. Therefore, it is important to update this value as soon as possible (we do this in our code for each node on the first time we visit it) and to traverse the most promising branches first. Of course, it is not easy to determine what the most promising branch is at any point. A good, although imperfect, indicator could be the distance between the target point and split line for a branch. The closer the target, the larger should be the intersection between the nearest neighbor perimeter (the hyper-sphere within which it’s still possible to find a closer point to the target), and the region on the other side of the split line. That’s the reason why we traverse the tree using a depth-first search: we backtrack on the smallest branches first, so hopefully when we reach larger branches close to the top of the tree, we can prune them.

9.3.6 *Region search*

While k-d trees were mainly designed to perform nearest neighbor search, they turned out to be particularly efficient for another kind of operation: querying the intersection of our dataset with a given region in the k-dimensional space.

In theory this region could be of any shape, but this operation becomes meaningful only if we can efficiently prune the branches we traverse during the query, and this heavily depends on the region’s morphology.

In practice, there are two main cases we are interested in:

- Hyper-spherical regions, as shown in figure 9.25; the geometric interpretation is to query points within a certain distance from a certain point of interest.
- Hyper-rectangular regions, as shown in figure 9.26; here, the geometric interpretation is to query points whose values are in certain ranges.

When we deal with spherical regions, we are in a similar situation as the nearest neighbor search: we need to include all the points within a certain distance from a given point. It’s like performing a NN-search, where we never update the distance to the nearest neighbor, and instead of keeping track of only one point (the NN), we gather all points closer than that distance.

In figure 9.25 you can see how we are going to prune branches. When we are at a split, we will certainly have to traverse the branch on the same side of the center of the search region P . For the other branch, we check the distance between P and its projection on the split line. If that distance is lower than or equal to the search region’s radius, it means that there is still an area of intersection between that branch and the search region, and so we need to traverse the branch; otherwise, we can prune it.

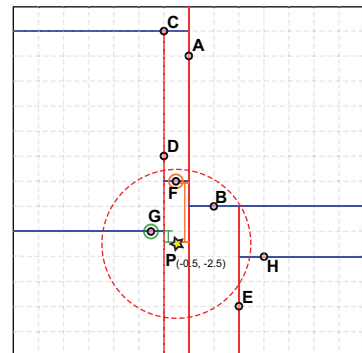
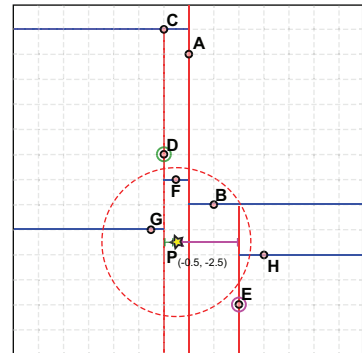
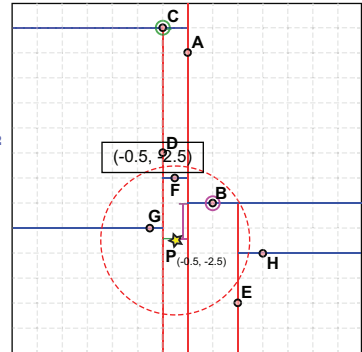
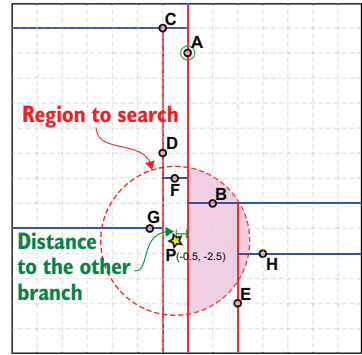
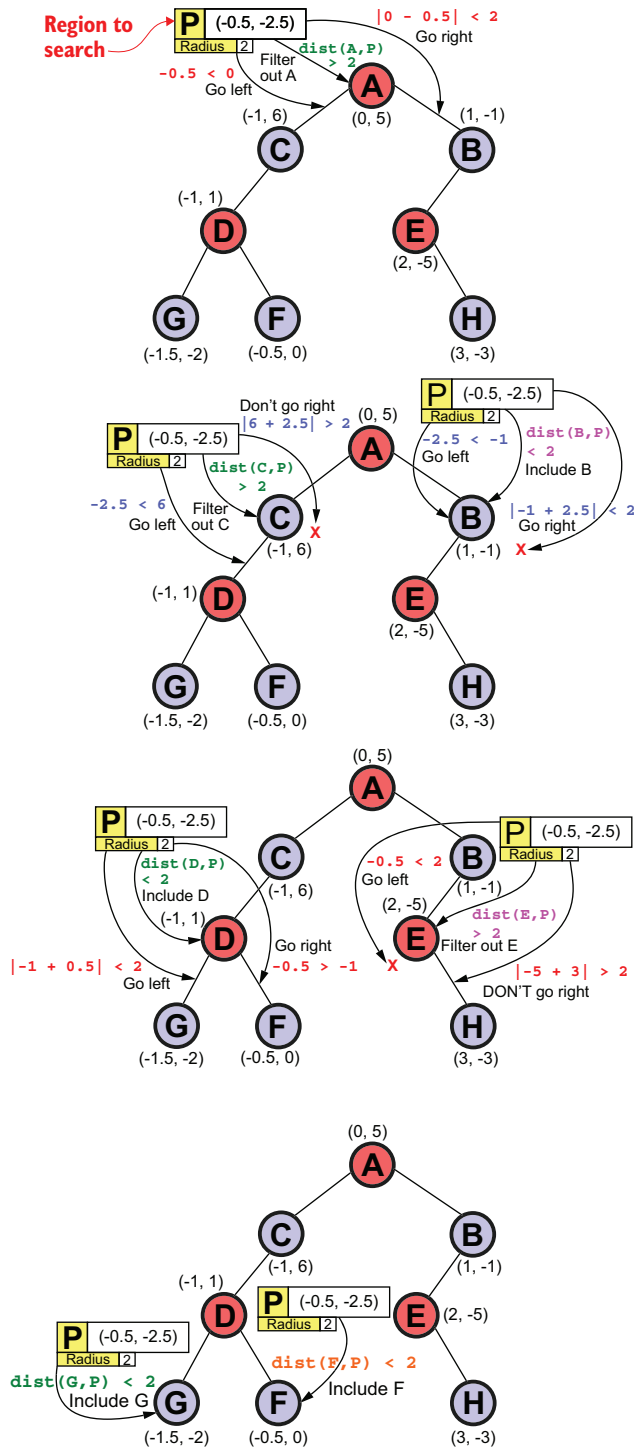
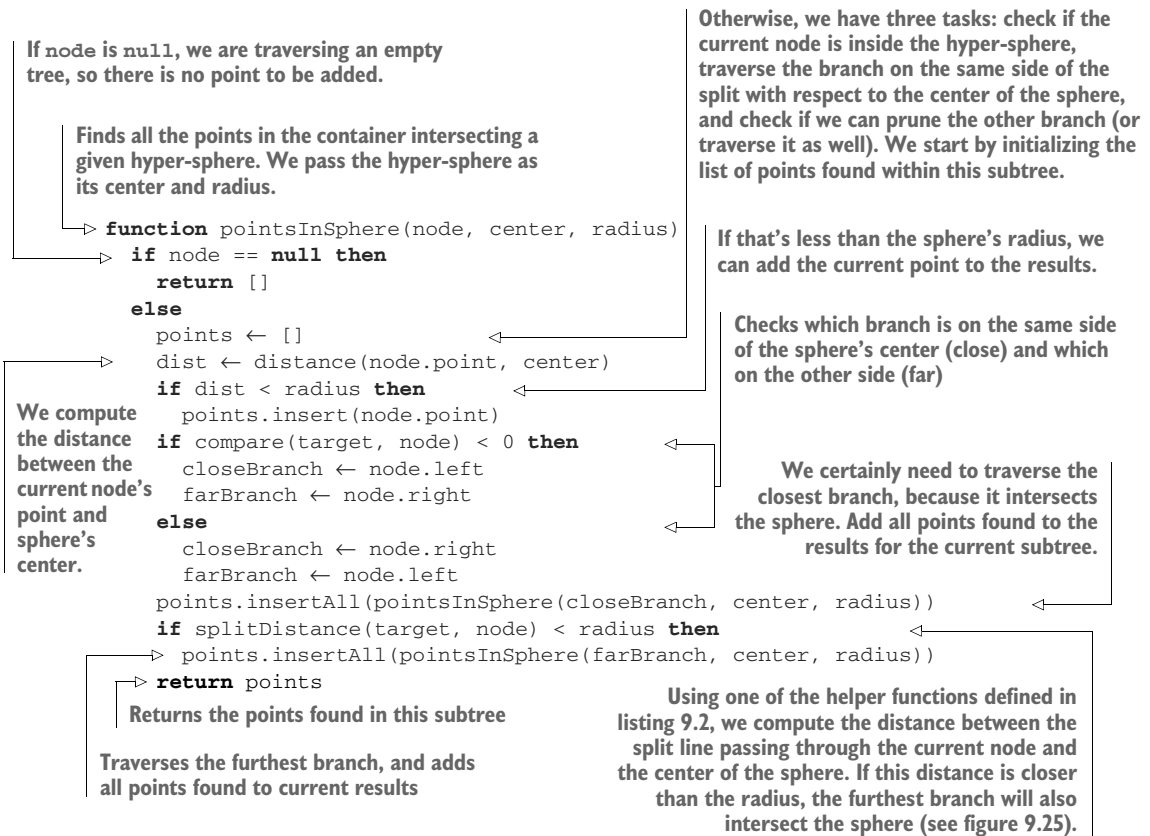


Figure 9.25 Region search on a k-d tree returns all the points in a k-d tree within a given hyper-sphere. This means looking for points within a given Euclidean distance from the sphere's center. We start our search from the root: it's not within the sphere. The point is on A's left branch, so we need to traverse it, but even if A is not within the sphere, the split line through it intersects the sphere, so there is a portion of the sphere intersecting the right branch of A as well (highlighted in the top-left figure). For the following steps, we are showing in parallel the execution on all branches at a given level, for the sake of space. (This is also a good hint that these processes could be executed in parallel.)

Listing 9.11 shows the pseudo-code for this method. As you can see, it's pretty similar to the regular NN-search.

Listing 9.11 The `pointsInSphere` method



The other region we are going to use for these queries is a rectangular shape. As you can imagine, the only difference with respect to `pointsinSphere` is the way we check whether we have to prune a branch. If we assume that the rectangle is oriented along the Cartesian axes used for splits, then pruning might even be considered easier, as shown in figure 9.26. But suppose we are at a horizontal split; then we need to understand if the split line intersects the search region, and in this case we will traverse both branches, or if it's above or below the region, which tells us which branch we can prune. This can be checked by simply comparing the y coordinate current node's point—call it N_y —with top (R_t) and bottom (R_b) y coordinates of the rectangular region, as we can have three cases:

- $R_b \leq N_y \leq R_t$ —We need to traverse both branches.
- $N_y > R_t$ —We can prune the left branch.
- $R_b > N_y$ —We can prune the right branch.

And similarly for vertical splits, by checking the x coordinates instead of the y coordinates, it can also be generalized for k -dimensional spaces, cycling through dimensions.

This method is particularly useful when we need to search values within simple boundaries for each feature in data. For instance, if we have a dataset containing the tenures and salaries of employees, we might want to search all employees who worked for the company between two and four years and have salaries between 40K and 80K . . . and give them a raise!¹⁴

This search, implemented in listing 9.12, translates into a rectangle with boundaries parallel to the dataset feature axes, meaning that in our boundaries each feature is independent of any other feature. If, instead, we had conditions that would mix more than one feature (for instance, a salary lower than 15K for each year of tenure), then the boundaries of the search region would be segments of generic lines, not parallel to any axis.

¹⁴In a perfect world . . .

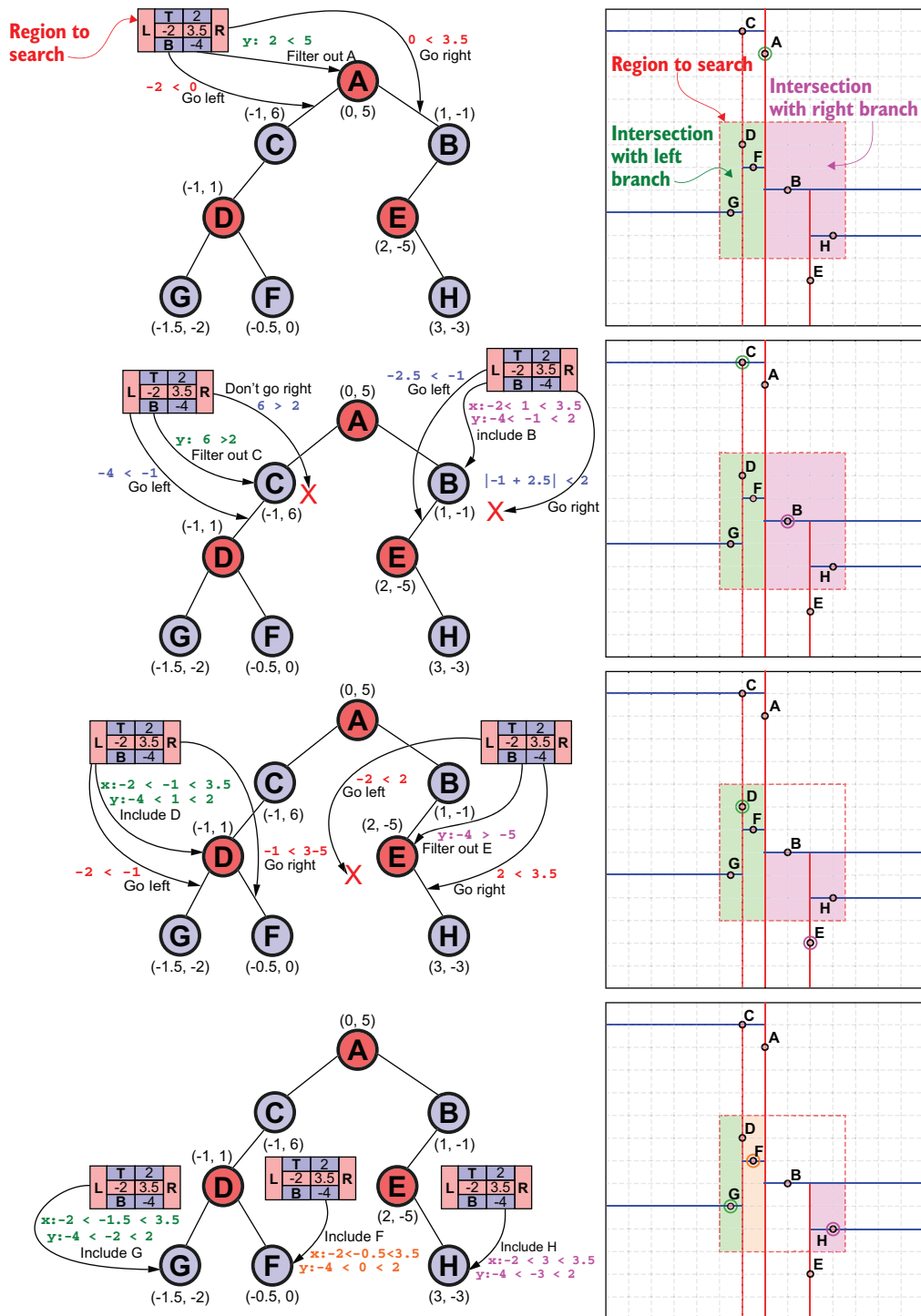


Figure 9.26 Region search on a k-d tree returns all the points in a k-d tree within a given hyper-rectangle. This means looking for points that, for each coordinate, satisfy two inequalities: each coordinate must be within a range. For instance, in this 2-D example, the points' x coordinates need to be between -2 and 3.5 , and y coordinates between -4 and 2 .

In that case, the problem becomes harder to solve, and we might need something more complex, such as the simplex algorithm,¹⁵ to search the points satisfying our range query.

Listing 9.12 The `pointsInRectangle` method

If the node is null, we are traversing an empty tree, so there is no point to be added.

Finds all the points in the container intersecting a given hypersphere. We pass the hyper-rectangle as an argument, so we can assume it is a list of named tuples, each containing the boundaries for a dimension of the rectangle, as a range (min-max values).

```

function pointsInRectangle(node, rectangle)
  if node == null then
    return []
  else
    points ← []
    if (rectangle[i].min ≤ node.point[i] ≤ rectangle[i].max
        ∀ 0 ≤ i < k) then
      points.insert(node.point)
      if intersectLeft(rectangle, node) then
        points.insertAll(pointsInRectangle(node.left, rectangle))
      if intersectRight(rectangle, node) then
        points.insertAll(pointsInRectangle(node.right, rectangle))
    return points

```

Finally, we return all points found in this subtree.

If the rectangular search region intersects the right branch, we can symmetrically recurse on the right child.

Otherwise, we have three tasks: check if the current node is inside the hyper-rectangle, check if the left and right branches intersect the rectangle (at least one of them will), and traverse any that do intersect.

We start by initializing the list of points found within this subtree.

If, for each dimension i , the i -th coordinate of the current node's point is within the rectangle's boundaries for that coordinate, then we can add the current point to the results.

If the rectangle boundaries intersect the left branch, either being on the left of the current node's split line, or intersecting it, then we need to traverse the left branch and add all points found to the current results. For the sake of space, we won't provide this helper function, but you can easily write it using figure 9.26 as a reference.

What's the performance of both region-searches? Well, as you can imagine, it heavily depends on the regions we search. We go on a full range from two edge cases:

¹⁵The simplex algorithm is an ingenious optimization method. It is not related to or helped by k-d trees and, as such, is out of scope for this chapter, but it's interesting reading, and you can read more here: https://en.wikipedia.org/wiki/Simplex_algorithm.

- For very small regions intersecting only a single branch corresponding to a leaf, we will prune every other branch, and just follow a path to the leaf, so the running time will be $O(h)$, where h is the height of the tree – $O(\log(n))$ for a balanced tree with n points.
- When the region is large enough to intersect all points, we will have to traverse the whole tree, and the running time will be $O(n)$.

Therefore, we can only say that the worst-case running time is $O(n)$, even if the methods will efficiently prune the tree whenever possible.

9.3.7 A recap of all methods

As we have seen, k-d trees provide a speed-up over brute-force search on the whole dataset; table 9.1 summarizes the performance of the methods described in this chapter. While the worst-case running time for nearest neighbor search (and removal) is still linear (exactly as for brute-force), in practice the amortized performance on balanced k-d trees is slightly to consistently better. The improvement is higher in low dimensional spaces and still consistent in medium-dimensional spaces.

In high-dimensional spaces, the exponential term on k for nearest neighbor becomes dominant and makes supporting the extra complexity of such a data structure not worth it.

Table 9.1 Operations provided by k-d tree, and their cost on a balanced k-d tree with n elements

Operation	Running time	Extra space
search	$O(\log(n))$	$O(1)$
insert	$O(\log(n))$	$O(1)$
remove	$O(n^{1-1/k})^a$	$O(1)$
findMin	$O(n^{1-1/k})^a$	$O(1)$
nearestNeighbor	$O(2^k + \log(n))^a$	$O(m)^b$
pointsInRegion	$O(n)$	$O(n)$

^aAmortized, for a k-d tree holding k -dimensional points.

^bWhen searching for the m nearest neighbors, with m constant (not a function of n).

9.4 Limits and possible improvements

If we look back at our “find the closest hub” problem, we started this chapter with basically nothing better than brute-force search to find the nearest neighbor of a point in a multi-dimensional dataset. Then, going through a few less-than-ideal attempts, we built our understanding of the traps and challenges in such a task and finally introduced k-d trees.